

TỔNG HỢP YÊU CẦU CÔNG NGHỆ THEO ĐỀ TÀI ĐÃ ĐĂNG KÝ

Bản hướng dẫn chi tiết dành cho sinh viên · Môn Các Công nghệ Lập trình Hiện đại · 09/2026

TRỌNG TÂM CỦA MÔN HỌC

Công nghệ là đối tượng học và đối tượng được đánh giá. Ứng dụng/đề tài nghiệp vụ chỉ là bối cảnh để nhóm chứng minh rằng mình hiểu và làm chủ công nghệ. Nghiệp vụ phức tạp không tự tạo thêm điểm; chiều sâu công nghệ mới quyết định chất lượng đồ án.

PHẠM VI VÀ CÁCH SỬ DỤNG TÀI LIỆU

- Tài liệu này tổng hợp yêu cầu cho các công nghệ đã đăng ký rõ ràng trong lớp. Các đề tài chỉ ghi “RAG”, “LLM”, “AI”, “chatbot” hoặc một kiến trúc chung mà chưa nêu thư viện/SDK/framework phải bổ sung công nghệ chính trước khi áp yêu cầu.
- Mỗi công nghệ được mô tả theo ba tầng. Tầng 1A là lõi bắt buộc; Tầng 1B là các cơ chế thuộc bản chất công nghệ nhưng nhóm tự chọn theo ứng dụng; Tầng 2 là chuẩn kỹ nghệ phần mềm; Tầng 3 là phần nâng cao/chuyên sâu.
- Không chấm cao vì dùng nhiều thư viện hoặc nhiều kỹ thuật. Một kỹ thuật chỉ có giá trị khi được dùng đúng, gắn với bài toán, nhóm giải thích được cơ chế và có minh chứng chạy/đo/kiểm thử.
- Nhóm không đăng ký “độ khó”. Sau khi nộp và khóa đồ án, bài thực hành tại chỗ được ra từ chính những gì nhóm đã làm. Làm được bài thực hành là cộng Đạt/Không đạt; điểm số sau đó được chấm từ chất lượng đồ án.
- Nếu ứng dụng không cần một tính năng Tầng 1B nào đó thì không phải cố nhét vào. Ngược lại, nếu nhóm đã khai hoặc đã dùng một cơ chế trong đồ án thì phải biết giải thích và chỉnh sửa nó.
- Miền nghiệp vụ nên đơn giản, quen thuộc, có thể dùng dữ liệu giả lập. Nếu nhóm phải dành nhiều thời gian học nghiệp vụ hơn học công nghệ thì đề tài đang lệch trọng tâm.

Quy tắc đọc nhanh

Tầng 1A trả lời: “Nếu bỏ những mục này thì còn có thể nói là em đã học công nghệ này không?”

Tầng 1B trả lời: “Ứng dụng của em cần thêm cơ chế nào của chính công nghệ?”

Tầng 2 trả lời: “Em có làm sản phẩm kỹ thuật một cách nghiêm túc không?”

Tầng 3 trả lời: “Em đã đào sâu công nghệ đến đâu, có đo lường/so sánh/phân tích gì vượt mức vận hành cơ bản?”

0. Tầng 2 — Chuẩn kỹ nghệ phần mềm dùng chung

Áp dụng theo đặc thù nền tảng. Không bắt một nền tảng no-code phải Docker hóa hay một ứng dụng managed-auth phải tự hash mật khẩu. Nhóm phải đạt yêu cầu tương đương phù hợp với công nghệ mình chọn.

- Quản lý phiên bản:** Dùng Git cho dự án có mã nguồn: nhánh làm việc, pull request/merge request, review chéo, lịch sử commit có ý nghĩa; không để toàn bộ nhóm cùng commit thẳng vào main.
- Khả năng chạy lại:** Có README rõ ràng, phiên bản runtime/package, lệnh cài và lệnh chạy. Backend/AI/self-host nên đóng gói Docker/docker compose khi hợp lý; frontend/mobile/no-code có thể dùng cơ chế tái tạo tương đương.
- Quản lý cấu hình và bí mật:** API key, mật khẩu, connection string phải qua biến môi trường/secret store; cung cấp .env.example nhưng không commit secret thật.
- Kiểm thử:** Có test phù hợp với sản phẩm: unit test cho logic; API/integration test cho backend; component/flow test cho frontend; test dữ liệu/đánh giá cho AI; test scenario cho low-code.
- CI và kiểm tra tự động:** Dự án code nên có GitHub Actions hoặc tương đương để build/lint/test. Nếu nền tảng không hỗ trợ CI kiểu code, phải có quy trình version/export/checklist thay thế.
- Bảo mật cơ bản:** Biết rủi ro liên quan trực tiếp tới đồ án: auth/authorization, input validation, CORS, injection, XSS, upload file, API key, quyền truy cập dữ liệu. Không cần “học thuộc OWASP” nếu không gắn với hệ thống.
- Xử lý lỗi và quan sát:** Không để lỗi rơi tự do. Có thông báo lỗi phù hợp; log đủ để truy được luồng chính; với hệ thống server nên có health check hoặc structured log khi phù hợp.
- Tài liệu kỹ thuật:** README phải giúp một nhóm khác chạy lại được. Nếu có API cần mô tả endpoint; nếu có mô hình/AI cần ghi model/config; nếu có dữ liệu cần mô tả nguồn và cách chuẩn bị.

- **Sử dụng AI hỗ trợ code:** Được phép dùng AI trong quá trình làm. Nhóm phải kiểm chứng code/claim do AI gợi ý và khai báo theo quy định môn học; không được dùng “AI đã viết” để thay cho khả năng giải thích.

1. Track Web Frontend

Định hướng track

Frontend không được đánh giá bằng số lượng trang hay mức độ “đẹp”. Nhóm phải chứng minh hiểu mô hình render, state/data flow, routing và cơ chế đặc thù của framework. Tầng 3 dành cho tương tác, hiệu năng, realtime, offline, đồ họa hoặc khả năng truy cập.

Next.js 16

Công nghệ chính: Next.js App Router. React là nền tảng bên dưới; yêu cầu phải thể hiện được những gì làm Next.js khác một React SPA thuần.

Tầng 1A — Lỗi bắt buộc

- App Router: tổ chức route theo thư mục; hiểu page/layout và cách cây route được dựng.
- Phân biệt Server Component và Client Component: biết component nào mặc định chạy phía server, khi nào cần “use client”, và tác động của ranh giới này tới dữ liệu/bundle.
- Lấy dữ liệu phía server trong Server Component; không biến mọi trang thành client component chỉ để gọi fetch.
- Hiểu ít nhất luồng render chính của chính đồ án: request vào route nào, dữ liệu lấy ở đâu, phần nào render server/phần nào cần tương tác client.

Tầng 1B — Lỗi tự chọn theo mục tiêu ứng dụng

Không có số lượng cứng. Nhóm chọn các mục thực sự cần cho ứng dụng; đã chọn thì phải hiểu cơ chế, cấu hình và kết quả.

- Route động/lồng nhau; layout dùng chung; loading/error/not-found phù hợp với luồng ứng dụng.
- Caching và revalidation: chọn chiến lược phù hợp dữ liệu; biết khi nào cần dữ liệu tươi và khi nào có thể cache.
- Server Action/Server Function cho mutation hoặc Route Handler khi cần API endpoint trong cùng dự án.
- Suspense/streaming để chia nhỏ thời điểm hiển thị dữ liệu chậm.
- Tối ưu Image/Font; metadata/SEO nếu mục tiêu sản phẩm cần.
- Middleware cho redirect/rewrite/check đơn giản; không lạm dụng middleware để chứa toàn bộ business logic.

Minh chứng tối thiểu cần có

- Chỉ ra ít nhất một route thật trong đồ án và giải thích server/client boundary của route đó.
- Có code và ảnh/log chứng minh data fetching hoạt động ở đúng phía đã thiết kế.
- Nếu dùng caching/revalidation hoặc Server Action/Route Handler, phải có tình huống chạy chứng minh trước/sau thay đổi.

Chưa được xem là đạt nếu chỉ...

- Chỉ tạo nhiều trang React rồi đặt vào thư mục app nhưng toàn bộ đều là Client Component.
- Dùng Next.js chỉ như router, trong khi dữ liệu vẫn được fetch hoàn toàn kiểu SPA và không giải thích được vì sao.
- Kể tên SSR/SSG/ISR nhưng đồ án không thể chỉ ra cơ chế render/caching nào đang thật sự dùng.

Câu hỏi tự kiểm

- “Cái gì trong đồ án này là Next.js mà một React SPA bình thường không làm giống như vậy?”
- “Nếu bỏ “use client” ở component này thì chuyện gì xảy ra? Tại sao?”
- “Dữ liệu này nên cache hay luôn lấy mới? Quyết định đó ảnh hưởng gì?”

Ví dụ phạm vi có thể kiểm tra tại chỗ

Đây chỉ là ví dụ. Đề chính thức được ra từ chính đồ án đã nộp, không hỏi kiến thức ngoài phạm vi nhóm đã làm.

- Thêm một dynamic route vào module có sẵn và xử lý not-found.
- Chuyển một thao tác cập nhật từ client fetch sang Server Action/Route Handler phù hợp với code hiện tại.

- Giải thích và chỉnh lại một server/client boundary chưa hợp lý trong chính đồ án.

React 19 + Vite + TanStack

Công nghệ chính là tổ hợp React + Vite + TanStack. Nếu nhóm chỉ dùng React cơ bản với useState/fetch thì chưa đạt đúng hồ sơ này.

Tầng 1A — Lỗi bắt buộc

- Component, props, hooks và luồng render/re-render; biết tránh đặt state dư thừa.
- Phân biệt client state (UI/local) với server state (dữ liệu từ backend).
- TanStack Query: query key, cache, trạng thái loading/error/success, stale data và invalidation sau mutation.
- Vite: hiểu vai trò dev server/build tool trong dự án và cách cấu hình cơ bản của dự án.

Tầng 1B — Lỗi tự chọn theo mục tiêu ứng dụng

Không có số lượng cứng. Nhóm chọn các mục thực sự cần cho ứng dụng; đã chọn thì phải hiểu cơ chế, cấu hình và kết quả.

- TanStack Router hoặc một router phù hợp để tổ chức route và route params.
- Mutation nâng cao: refetch có chủ đích, optimistic update, rollback khi lỗi.
- Error boundary/Suspense phù hợp với thư viện và kiến trúc đang dùng.
- Code splitting/lazy loading và phân tích bundle build của Vite.
- Bật React Compiler nếu môi trường hỗ trợ và đo/quan sát lại render bằng profiler; không chỉ “bật cho có”.
- Form library/validation nếu ứng dụng có biểu mẫu phức tạp.

Minh chứng tối thiểu cần có

- Chỉ ra một dữ liệu server state và giải thích vòng đời cache của nó.
- Có ít nhất một mutation làm thay đổi dữ liệu và xử lý cache sau mutation.
- Có build production chạy được; nếu có tối ưu bundle/render phải kèm số liệu hoặc profiler.

Chưa được xem là đạt nếu chỉ...

- Chỉ dùng React component + useState + fetch thuần, không có phần TanStack đúng nghĩa.
- Gọi lại API sau mọi thao tác nhưng không hiểu cache/invalidation.
- Dùng thư viện state/query chỉ vì mẫu code AI sinh ra nhưng không giải thích được state thuộc loại nào.

Câu hỏi tự kiểm

- “Dữ liệu nào trong app là client state, dữ liệu nào là server state?”
- “Khi mutation thành công, vì sao query này cần invalidate/refetch?”
- “Nếu bỏ TanStack Query, nhóm sẽ phải tự quản lý những phần nào?”

Ví dụ phạm vi có thể kiểm tra tại chỗ

Đây chỉ là ví dụ. Đề chính thức được ra từ chính đồ án đã nộp, không hỏi kiến thức ngoài phạm vi nhóm đã làm.

- Thêm một mutation mới và cập nhật cache đúng cách.
- Thay query key để hỗ trợ bộ lọc/phân trang và giải thích cache tách theo tham số.
- Phân tích một re-render không cần thiết bằng profiler và sửa nếu nhóm đã làm phần hiệu năng.

Angular

Trọng tâm là kiến trúc component–service, dependency injection và hệ sinh thái Angular; không phải chỉ dựng giao diện bằng template.

Tầng 1A — Lỗi bắt buộc

- Standalone component; input/output hoặc cơ chế giao tiếp component phù hợp.
- Dependency Injection và service: biết vì sao logic/dữ liệu được đưa vào service thay vì dồn trong component.
- Router: route, tham số và ít nhất một guard hoặc cơ chế bảo vệ điều hướng khi ứng dụng có auth/quyền.

Tầng 1B — Lỗi tự chọn theo mục tiêu ứng dụng

Không có số lượng cứng. Nhóm chọn các mục thực sự cần cho ứng dụng; đã chọn thì phải hiểu cơ chế, cấu hình và kết quả.

- Reactive Forms với validation có ý nghĩa.
- Signals cho state phù hợp; phân biệt khi nào dùng signal và khi nào dùng observable.
- RxJS cho luồng async/phối hợp sự kiện; tránh subscribe lồng nhau thiếu kiểm soát.
- HttpClient + interceptor cho token/log/error chung.
- Lazy loading để tách feature lớn.
- Change detection strategy và đo/giải thích khi ứng dụng có yêu cầu hiệu năng.

Minh chứng tối thiểu cần có

- Cây component/service rõ ràng; chỉ ra service được inject ở đâu.
- Có route thật và guard/validation hoạt động nếu đã chọn.
- Nếu dùng RxJS/signals, có ví dụ data flow trong code chứ không chỉ import thư viện.

Chưa được xem là đạt nếu chỉ...

- Đặt toàn bộ logic gọi API và xử lý dữ liệu trong một component lớn.
- Dùng service như một lớp “cho có” nhưng component vẫn phụ thuộc trực tiếp mọi thứ.
- Dùng RxJS rất nhiều operator nhưng không giải thích được luồng observable.

Câu hỏi tự kiểm

- “Vì sao chỗ này dùng service chứ không đặt logic thẳng trong component?”
- “Guard của em bảo vệ điều gì? Nếu người dùng đi thẳng vào URL thì sao?”
- “State này phù hợp signal hay observable hơn? Vì sao?”

Ví dụ phạm vi có thể kiểm tra tại chỗ

Đây chỉ là ví dụ. Đề chính thức được ra từ chính đề án đã nộp, không hỏi kiến thức ngoài phạm vi nhóm đã làm.

- Thêm một route có guard và thông báo điều hướng phù hợp.
- Tách một khối logic lớn khỏi component sang service và giải thích dependency.
- Thêm validation bất đồng bộ hoặc xử lý lỗi HTTP tập trung nếu đề án có cơ chế này.

Nuxt 4 (Vue 3)

Trọng tâm là file-based routing, SSR/hydration và data fetching theo mô hình Nuxt. Vue 3 là nền tảng phía dưới.

Tầng 1A — Lỗi bắt buộc

- Routing theo tệp và layout/page cơ bản.
- Hiểu SSR và hydration trong chính ứng dụng: phần nào được render trên server, phần nào tiếp quản ở client.
- useFetch hoặc useAsyncData đúng ngữ cảnh; xử lý loading/error/data rõ ràng.

Tầng 1B — Lỗi tự chọn theo mục tiêu ứng dụng

Không có số lượng cứng. Nhóm chọn các mục thực sự cần cho ứng dụng; đã chọn thì phải hiểu cơ chế, cấu hình và kết quả.

- Composable tự viết để tái sử dụng logic.
- Route middleware cho auth/redirect/rule đơn giản.
- Server route của Nitro khi ứng dụng cần endpoint phía server.
- Route rules/caching theo route nếu phù hợp.
- Chia sẻ state bằng useState/Pinia nếu ứng dụng thực sự có state dùng chung.
- Error page/handling và SEO/meta khi phù hợp.

Minh chứng tối thiểu cần có

- Có ít nhất một trang lấy dữ liệu theo cơ chế Nuxt và giải thích SSR/hydration.
- Chỉ ra route tạo từ cấu trúc file và data flow của route đó.
- Nếu dùng server route/middleware/composable, phải chỉ ra nơi được gọi và lý do dùng.

Chưa được xem là đạt nếu chỉ...

- Dùng Nuxt như Vue SPA thông thường nhưng không giải thích được SSR/hydration.
- Dùng onMounted + fetch cho mọi trang dù có thể lấy dữ liệu bằng cơ chế Nuxt.
- Kể tên Nitro/middleware nhưng đề án không có ví dụ thực sự.

Câu hỏi tự kiểm

- “Trang này render gì ở server? Sau hydration điều gì thay đổi?”
- “Vì sao dùng useFetch/useAsyncData ở đây thay vì onMounted?”
- “Route middleware này khác server middleware ở điểm nào trong đồ án?”

Ví dụ phạm vi có thể kiểm tra tại chỗ

Đây chỉ là ví dụ. Đề chính thức được ra từ chính đồ án đã nộp, không hỏi kiến thức ngoài phạm vi nhóm đã làm.

- Thêm một dynamic page dùng useAsyncData và xử lý lỗi.
- Viết một composable dùng lại ở hai trang hiện có.
- Chuyển một API đơn giản vào Nitro server route nếu kiến trúc đồ án đã dùng server side.

SvelteKit

Trọng tâm là mô hình reactivity của Svelte, routing/load và ranh giới server–client của SvelteKit.

Tầng 1A — Lỗi bắt buộc

- Reactivity của Svelte và luồng cập nhật UI; biết state nào là local, state nào cần dùng chung.
- Hàm load trong route và cách dữ liệu đi vào page.
- Phân biệt code chạy server và code chạy client; tránh dùng browser API ở server hoặc lộ secret sang client.

Tầng 1B — Lỗi tự chọn theo mục tiêu ứng dụng

Không có số lượng cứng. Nhóm chọn các mục thực sự cần cho ứng dụng; đã chọn thì phải hiểu cơ chế, cấu hình và kết quả.

- Form actions cho mutation/form submit.
- SSR hoặc prerender theo loại trang.
- Hooks cho xử lý chung (ví dụ auth/context).
- Stores/context nếu có state dùng chung.
- Streaming/deferred data nếu ứng dụng có dữ liệu chậm.
- Adapter/deployment target nếu nhóm muốn nghiên cứu sâu khâu build/deploy.

Minh chứng tối thiểu cần có

- Có ít nhất một route dùng load và một tương tác làm thay đổi state/data.
- Chỉ ra rõ file/module nào chỉ chạy server và tại sao.
- Nếu dùng form action/hook/store, phải có luồng sử dụng thật.

Chưa được xem là đạt nếu chỉ...

- Chỉ viết component Svelte nhưng không dùng cơ chế của SvelteKit.
- Đưa secret/API key vào code client.
- Không phân biệt được dữ liệu nào được load ở server và dữ liệu nào sinh ở browser.

Câu hỏi tự kiểm

- “Hàm load này chạy ở đâu và khi nào?”
- “Nếu đoạn code này dùng window/localStorage thì có chạy được ở server không?”
- “Form action giải quyết vấn đề gì so với fetch thủ công?”

Ví dụ phạm vi có thể kiểm tra tại chỗ

Đây chỉ là ví dụ. Đề chính thức được ra từ chính đồ án đã nộp, không hỏi kiến thức ngoài phạm vi nhóm đã làm.

- Thêm một form action vào một luồng có sẵn.
- Chuyển một trang tĩnh sang prerender/SSR phù hợp và giải thích.
- Sửa một lỗi do dùng browser-only API sai phía.

LƯU Ý — VUE.JS KHÔNG KÈM NUXT

Danh mục chính thức hiện dùng Nuxt 4 cho hệ sinh thái Vue. Nhóm chỉ dùng Vue + Vite cần trao đổi với giảng viên để chốt lại công nghệ chính và yêu cầu tương ứng. Không tự mặc định rằng một Vue SPA cơ bản được xem là tương đương Nuxt.

Tầng 3 — Nâng cao chung cho Web Frontend

Chỉ chọn khi phù hợp. Tầng 3 không phải danh sách “phải đủ 2 mục”; độ sâu và minh chứng quan trọng hơn số lượng.

- **Kéo thả:** Xử lý drag/drop có state thật, lưu được thứ tự hoặc cấu trúc sau thao tác; không chỉ demo kéo một ô màu.
- **Hoạt hoặ:** CSS View Transitions/Motion/GSAP... dùng có mục đích; nếu nói tối ưu phải quan sát bundle hoặc trải nghiệm chứ không chỉ thêm hiệu ứng.
- **3D/Canvas/Data visualization:** Three.js/R3F/Konva/D3; cần có tương tác hoặc dữ liệu thật của demo, đồng thời kiểm soát hiệu năng.
- **Danh sách lớn/virtualization:** Dùng virtual list cho dữ liệu đủ lớn để thấy khác biệt; có so sánh render/scroll trước-sau.
- **Realtime/cộng tác:** WebSocket/SSE/CRDT; cần xử lý kết nối, cập nhật trạng thái, reconnect hoặc conflict ở mức phù hợp.
- **Rich editor:** TipTap/Lexical/Slate; cần dữ liệu có cấu trúc, lưu/khôi phục nội dung, không chỉ dùng editor mặc định.
- **PWA/offline:** Service Worker/IndexedDB; phải chứng minh ứng dụng vẫn xử lý được một luồng khi mất mạng và đồng bộ lại khi có mạng.
- **Accessibility/performance:** Kiểm tra keyboard/focus/ARIA hoặc Lighthouse/Core Web Vitals theo phạm vi; có số liệu và sửa ít nhất một vấn đề cụ thể.

2. Track Backend & API

Định hướng track

Backend được đánh giá bằng cách tổ chức request flow, validation, data access, auth, error handling, performance và khả năng vận hành. CRUD nhiều bảng không tự tạo chiều sâu.

NestJS

Trọng tâm là kiến trúc module và dependency injection của NestJS, cùng các cơ chế xử lý request như Pipe/Guard/Interceptor/Filter khi ứng dụng cần.

Tầng 1A — Lỗi bắt buộc

- Module: biết feature thuộc module nào, module import/export provider ra sao.
- Controller: route/HTTP method, nhận input và giao việc đúng cho service/provider.
- Provider + Dependency Injection: inject dependency qua constructor, hiểu vì sao không tự new service thủ công.
- Luồng request tối thiểu controller → provider/service → data source, kèm error response rõ ràng.

Tầng 1B — Lỗi tự chọn theo mục tiêu ứng dụng

Không có số lượng cứng. Nhóm chọn các mục thực sự cần cho ứng dụng; đã chọn thì phải hiểu cơ chế, cấu hình và kết quả.

- DTO + ValidationPipe để chuẩn hóa/kiểm tra input.
- Guard cho authentication/authorization.
- Interceptor cho logging/transform/cache/timing phù hợp.
- Exception Filter cho lỗi tập trung.
- Custom decorator để lấy user/role/metadata gọn hơn.
- Dynamic module/config module khi dự án cần cấu hình linh hoạt.
- ORM/query layer và transaction nếu ứng dụng có CSDL.

Minh chứng tối thiểu cần có

- Sơ đồ module hoặc mô tả request flow của một endpoint thật.
- Chỉ ra ít nhất một dependency được inject và vòng đời phù hợp.
- Nếu dùng Guard/Pipe/Interceptor/Filter, có request mẫu chứng minh trước/sau khi cơ chế chạy.

Chưa được xem là đạt nếu chỉ...

- Chỉ có controller + service CRUD, không giải thích được module/DI.

- Tự new service/repository bên trong controller.
- Đặt kiểm tra quyền trong mọi controller bằng if/else lặp lại nhưng vẫn khai đã học Guard.

Câu hỏi tự kiểm

- “Guard và Interceptor khác nhau ở đâu, và vì sao em đặt kiểm tra quyền ở Guard?”
- “Provider này được tạo/inject ở scope nào?”
- “Input sai được chặn ở đâu trước khi vào business logic?”

Ví dụ phạm vi có thể kiểm tra tại chỗ

Đây chỉ là ví dụ. Đề chính thức được ra từ chính đề án đã nộp, không hỏi kiến thức ngoài phạm vi nhóm đã làm.

- Thêm một Guard/role rule vào module hiện có.
- Viết một Pipe hoặc DTO validation cho trường dữ liệu mới.
- Thêm Interceptor đo thời gian/chuẩn hóa response nếu đề án đã dùng pattern này.

ASP.NET Core

Trọng tâm là middleware pipeline, dependency injection và cách endpoint được tổ chức trong ASP.NET Core.

Tầng 1A — Lỗi bắt buộc

- Middleware pipeline và thứ tự xử lý request/response.
- DI tích hợp của framework; biết khác nhau cơ bản giữa singleton/scoped/transient trong chính dự án.
- Controller hoặc Minimal API: chọn một cách phù hợp, route/model binding rõ ràng.
- Xử lý lỗi/HTTP status code nhất quán.

Tầng 1B — Lỗi tự chọn theo mục tiêu ứng dụng

Không có số lượng cứng. Nhóm chọn các mục thực sự cần cho ứng dụng; đã chọn thì phải hiểu cơ chế, cấu hình và kết quả.

- EF Core + migration, quan hệ và truy vấn hợp lý.
- Authentication/authorization bằng policy/role/claim.
- Model binding và validation.
- Filter khi cần xử lý cross-cutting concern ở MVC/controller level.
- Hosted service/background service cho tác vụ nền.
- Configuration theo môi trường và options pattern.
- Health checks/logging.

Minh chứng tối thiểu cần có

- Chỉ ra thứ tự middleware và một tình huống thứ tự đó ảnh hưởng kết quả.
- Có service được inject thật, không khởi tạo thủ công.
- Nếu dùng EF/auth/hosted service, phải có migration/request/log minh chứng.

Chưa được xem là đạt nếu chỉ...

- Chỉ dùng template mặc định rồi viết CRUD controller.
- Không phân biệt middleware với filter.
- Đăng ký service sai lifetime nhưng không giải thích được hậu quả.

Câu hỏi tự kiểm

- “Thứ tự các middleware trong pipeline của em có ý nghĩa gì? Đảo hai cái thì sao?”
- “Service này nên scoped hay singleton? Vì sao?”
- “Authorization của endpoint này được quyết định ở đâu?”

Ví dụ phạm vi có thể kiểm tra tại chỗ

Đây chỉ là ví dụ. Đề chính thức được ra từ chính đề án đã nộp, không hỏi kiến thức ngoài phạm vi nhóm đã làm.

- Chèn một middleware mới và chứng minh vị trí đặt đúng.
- Thêm một policy authorization cho endpoint có sẵn.
- Thêm migration và cập nhật API theo schema mới.

FastAPI

Trọng tâm là type-driven API với Pydantic, dependency system và async I/O đúng chỗ; không phải cứ viết async def là “bất đồng bộ”.

Tầng 1A — Lỗi bắt buộc

- Pydantic model cho request/response hoặc schema dữ liệu; hiểu validation tự động.
- Dependency system (`Depends`) cho dependency dùng chung như auth, DB session, config.
- Dùng `async` đúng chỗ: endpoint I/O-bound có awaitable operation; biết khi nào không cần async.
- HTTP status/error response rõ ràng.

Tầng 1B — Lỗi tự chọn theo mục tiêu ứng dụng

Không có số lượng cứng. Nhóm chọn các mục thực sự cần cho ứng dụng; đã chọn thì phải hiểu cơ chế, cấu hình và kết quả.

- Response model và schema OpenAPI có chủ đích.
- Exception handler tập trung.
- Background task cho công việc ngăn sau response; phân biệt với queue thật.
- Middleware cho logging/CORS/timing.
- Lifespan/startup-shutdown để quản lý resource/model connection.
- TestClient/httpx cho API test.
- Dependency override phục vụ test.

Minh chứng tối thiểu cần có

- OpenAPI/docs chạy được và đúng schema.
- Chỉ ra dependency graph của ít nhất một endpoint.
- Có ví dụ endpoint async và giải thích await đang chờ I/O gì.

Chưa được xem là đạt nếu chỉ...

- Viết mọi endpoint `async def` nhưng bên trong toàn code blocking.
- Không dùng Pydantic/Depends mà xử lý dict/input thủ công.
- Dùng BackgroundTasks cho công việc dài/hàng đợi nặng nhưng không hiểu giới hạn.

Câu hỏi tự kiểm

- “Endpoint nào của em nên là async và endpoint nào không? Vì sao?”
- “Dependency này được tạo ở đâu và dùng lại thế nào?”
- “Nếu model input sai kiểu, lỗi được sinh ra ở tầng nào?”

Ví dụ phạm vi có thể kiểm tra tại chỗ

Đây chỉ là ví dụ. Đề chính thức được ra từ chính đề án đã nộp, không hỏi kiến thức ngoài phạm vi nhóm đã làm.

- Thêm dependency auth/config vào một endpoint hiện có.
- Thêm response model/validation cho một luồng thật.
- Chuyển một đoạn blocking không phù hợp ra cách xử lý đúng hơn nếu đề án có.

Spring Boot

Trọng tâm là IoC/DI, tầng web-service-data và hệ sinh thái Spring; không phải chỉ dùng annotation để tạo CRUD.

Tầng 1A — Lỗi bắt buộc

- IoC/Dependency Injection: bean được tạo và inject thế nào.
- REST controller và tầng service tách trách nhiệm.
- Spring Data JPA/repository cho data access; hiểu entity/relationship cơ bản.
- Exception/HTTP response có cấu trúc.

Tầng 1B — Lỗi tự chọn theo mục tiêu ứng dụng

Không có số lượng cứng. Nhóm chọn các mục thực sự cần cho ứng dụng; đã chọn thì phải hiểu cơ chế, cấu hình và kết quả.

- Bean Validation cho input.
- Configuration/profile theo môi trường.

- Spring Security cho auth/authorization.
- AOP cho cross-cutting concern phù hợp.
- Global exception handler.
- Test slice (`@WebMvcTest`, `@DataJpaTest`) hoặc integration test.
- Transaction cho nhiều thao tác dữ liệu cần nhất quán.

Minh chứng tối thiểu cần có

- Chỉ ra bean/service/repository được Spring quản lý và cách inject.
- Có truy vấn JPA/repository thật và migration/schema tương ứng.
- Nếu dùng Security/Transaction/AOP, phải có tình huống chứng minh.

Chưa được xem là đạt nếu chỉ...

- Chỉ scaffold entity/repository/controller CRUD.
- Tạo dependency bằng `new` thay vì để container quản lý.
- Dùng annotation nhưng không giải thích được lúc nào framework gọi.

Câu hỏi tự kiểm

- “Bean này do ai tạo? Nếu có hai implementation thì inject cái nào?”
- “Transaction bắt đầu/kết thúc ở đâu trong luồng này?”
- “Controller và service của em khác nhau về trách nhiệm gì?”

Ví dụ phạm vi có thể kiểm tra tại chỗ

Đây chỉ là ví dụ. Đề chính thức được ra từ chính đề án đã nộp, không hỏi kiến thức ngoài phạm vi nhóm đã làm.

- Thêm validation + global exception response cho một endpoint.
- Thêm một policy/role trong Spring Security nếu đề án dùng auth.
- Viết một test slice cho controller/repository hiện có.

Express.js

Express nhẹ và linh hoạt; chiều sâu phải đến từ việc nhóm tự thiết kế middleware, error flow, validation, data access và các năng lực server cần cho ứng dụng.

Tầng 1A — Lỗi bắt buộc

- Middleware pipeline và thứ tự middleware.
- Router/route params/query/body rõ ràng.
- Xử lý lỗi bất đồng bộ tập trung; tránh unhandled promise rejection hoặc try/catch lặp lại.
- Input validation cơ bản và HTTP status phù hợp.

Tầng 1B — Lỗi tự chọn theo mục tiêu ứng dụng

Không có số lượng cứng. Nhóm chọn các mục thực sự cần cho ứng dụng; đã chọn thì phải hiểu cơ chế, cấu hình và kết quả.

- Validation library/schema.
- Helmet/rate limit/CORS theo threat model thật.
- Authentication/authorization.
- Data access layer + migration/schema.
- Cấu trúc phân lớp phù hợp quy mô (router/controller/service/repository nếu cần).
- API test và mock dependency.
- Upload/streaming file nếu ứng dụng cần.

Minh chứng tối thiểu cần có

- Sơ đồ request qua các middleware quan trọng.
- Một ví dụ lỗi async đi qua error handler.
- Nếu dùng auth/security/data layer, có test/request chứng minh.

Chưa được xem là đạt nếu chỉ...

- Chỉ CRUD in-memory với vài route và không có validation/error/security.
- Dùng middleware nhưng không hiểu thứ tự.
- Đưa toàn bộ logic vào route handler dài hàng trăm dòng.

Câu hỏi tự kiểm

- “Request này đi qua những middleware nào theo thứ tự?”
- “Nếu service throw lỗi async thì ai bắt?”
- “Express không ép kiến trúc; tại sao nhóm chọn cấu trúc thư mục này?”

Ví dụ phạm vi có thể kiểm tra tại chỗ

Đây chỉ là ví dụ. Đề chính thức được ra từ chính đề án đã nộp, không hỏi kiến thức ngoài phạm vi nhóm đã làm.

- Thêm middleware auth/rate-limit/logging vào flow có sẵn.
- Thêm validation schema cho endpoint và chuẩn hóa lỗi.
- Chuyển một route handler quá lớn thành các lớp/module hợp lý.

CẢNH BÁO — EXPRESS.JS RẤT NHẸ

Nếu đề án chỉ dừng ở CRUD cơ bản thì khối lượng công nghệ không đủ. Nhóm phải đào sâu bằng các nhu cầu thật của backend như auth, cache, queue, realtime, search, streaming, observability hoặc kiểm thử. Không có số lượng bắt buộc cứng; giảng viên xem độ sâu thực tế.

Django

Tầng 1A — Lỗi bắt buộc

- Django ORM: model, migration, QuerySet và quan hệ dữ liệu.
- URL routing + view; hiểu request/response flow.
- Django Admin đúng bản chất để quản trị dữ liệu nội bộ; không tự viết lại một CRUD admin chỉ để tăng số trang.
- Một hướng giao diện rõ: Django Templates hoặc Django REST Framework nếu xây API.

Tầng 1B — Lỗi tự chọn theo mục tiêu ứng dụng

Không có số lượng cứng. Nhóm chọn các mục thực sự cần cho ứng dụng; đã chọn thì phải hiểu cơ chế, cấu hình và kết quả.

- DRF serializers/viewset/router nếu làm API.
- Django Forms/ModelForm nếu server-rendered form.
- Auth/permission qua `django.contrib.auth` hoặc DRF permission.
- Middleware tùy chỉnh.
- Signals khi thật sự cần phản ứng theo model event.
- Cache tích hợp/Redis.
- Celery/RQ nếu có background job.

Minh chứng tối thiểu cần có

- Migration và truy vấn QuerySet thật.
- Admin hoặc DRF chạy được đúng vai trò.
- Nếu dùng signal/cache/task queue, phải giải thích được trigger và failure case.

Chưa được xem là đạt nếu chỉ...

- Chỉ dùng Django như một router rồi viết SQL/CRUD thủ công.
- Tự viết lại hệ thống auth/admin mà không có lý do học thuật/kỹ thuật.
- Dùng signal cho mọi thứ khiến luồng khó truy vết.

Câu hỏi tự kiểm

- “Tại sao dùng QuerySet/ORM thay vì tự ghép SQL ở đây?”
- “Admin của Django dùng cho ai và khác giao diện người dùng cuối thế nào?”
- “Signal này có thật sự cần hay có thể gọi trực tiếp trong service/view?”

Ví dụ phạm vi có thể kiểm tra tại chỗ

Đây chỉ là ví dụ. Đề chính thức được ra từ chính đề án đã nộp, không hỏi kiến thức ngoài phạm vi nhóm đã làm.

- Thêm migration + query/filter phù hợp.
- Thêm permission cho một DRF endpoint hoặc một form flow.

- Tối ưu một truy vấn N+1 bằng select_related/prefetch_related nếu đồ án có.

Tầng 3 — Nâng cao chung cho Backend & API

Các mục này chỉ có giá trị khi giải quyết một nhu cầu kỹ thuật thật trong đồ án và có minh chứng/đo lường.

- **Cache và tối ưu truy vấn:** Redis/Valkey, TTL/invalidation, index CSDL, cursor pagination; nên có số liệu latency/query count trước-sau.
- **Hàng đợi và xử lý nền:** Job chậm đưa vào queue; có retry/backoff, trạng thái job, idempotency hoặc chống tạo trùng.
- **Realtime server:** WebSocket/SSE; có auth, lifecycle kết nối, broadcast/room hoặc reconnect phù hợp.
- **Tim kiếm nâng cao:** Full-text search/vector search khi đúng bài toán; có index và đánh giá kết quả, không chỉ `LIKE %keyword%`.
- **Tích hợp bên thứ ba:** Thanh toán/email/storage/webhook; có xác thực webhook, xử lý lỗi mạng, retry và tránh xử lý lặp.
- **Streaming/dữ liệu lớn:** Đọc/ghi theo stream, chunk/batch và progress; chứng minh không giữ toàn bộ file lớn trong RAM nếu mục tiêu là streaming.
- **Observability:** Structured log, trace/request id, metrics/health; có thể truy lại một request lỗi xuyên các tầng.
- **Kiến trúc nhiều dịch vụ:** Chỉ dùng khi thật sự có lý do; phải giải thích boundary, communication và failure. Không chia microservice chỉ để “trông nâng cao”.

3. Track Ứng dụng AI

Định hướng track

Track AI không chấm cao vì “có AI”. Nhóm phải chỉ ra pipeline kỹ thuật, dữ liệu vào/ra, cấu hình mô hình, lỗi/giới hạn và cách đánh giá bằng số hoặc test case. UI chỉ là lớp demo.

Ultralytics YOLO — hồ sơ Computer Vision

Công nghệ chính: Ultralytics YOLO cho bài toán detection/segmentation/tracking phù hợp. Không biến đề tài thành “gọi model rồi vẽ bounding box”.

Tầng 1A — Lỗi bắt buộc

- Bộ dữ liệu: nguồn, lớp nhãn, format annotation và cách chia train/validation/test hoặc ít nhất evaluation set.
- Chạy inference và giải thích output chính (box/class/confidence; mask nếu segmentation).
- Đánh giá bằng precision, recall và mAP phù hợp; không chỉ báo accuracy chung chung.
- Hiểu confidence threshold và NMS ở mức có thể giải thích vì sao một box bị giữ/loại.

Tầng 1B — Lỗi tự chọn theo mục tiêu ứng dụng

Không có số lượng cứng. Nhóm chọn các mục thực sự cần cho ứng dụng; đã chọn thì phải hiểu cơ chế, cấu hình và kết quả.

- Tiền xử lý/resizing/augmentation phù hợp dữ liệu.
- Fine-tune trên dữ liệu riêng nếu pretrained model chưa phù hợp.
- Model serving qua FastAPI/HTTP hoặc tích hợp trực tiếp vào app tùy kiến trúc.
- Xuất ONNX hoặc format khác khi mục tiêu là tối ưu triển khai.
- Object tracking nếu ứng dụng cần theo vết qua khung hình.
- Benchmark latency/FPS trên CPU/GPU hoặc thiết bị đích.
- Phân tích lỗi theo lớp, kích thước đối tượng, điều kiện ánh sáng.

Minh chứng tối thiểu cần có

- Có dữ liệu/nhãn mà nhóm giải thích được.
- Có bảng metric trên tập đánh giá và ít nhất vài ví dụ đúng/sai.
- Nếu triển khai thành service, có request/response và đo độ trễ.

Chưa được xem là đạt nếu chỉ...

- Chỉ dùng model pretrained trên vài ảnh mẫu Internet và chụp screenshot bounding box.
- Không có ground truth nhưng vẫn tự kết luận “độ chính xác cao”.

- Chỉ thay confidence bằng thử tay mà không đo ảnh hưởng.

Câu hỏi tự kiểm

- “mAP của nhóm được tính trên tập nào?”
- “Tăng confidence threshold sẽ ảnh hưởng precision/recall thế nào trong dữ liệu này?”
- “Một lỗi false positive cụ thể đến từ đâu?”

Ví dụ phạm vi có thể kiểm tra tại chỗ

Đây chỉ là ví dụ. Đề chính thức được ra từ chính đề án đã nộp, không hỏi kiến thức ngoài phạm vi nhóm đã làm.

- Thêm một class/dữ liệu mới và chạy lại evaluation.
- Thay threshold/NMS config và giải thích thay đổi output.
- Đo FPS/latency ở hai cấu hình nếu nhóm đã làm performance.

MCP SDK — hồ sơ MCP Server

MCP là giao thức/chuẩn kết nối AI với tools/resources. Mục tiêu là xây server có capability rõ, client thật gọi được, input schema và quyền được kiểm soát.

Tầng 1A — Lỗi bắt buộc

- Dựng MCP server bằng SDK đã chọn và giải thích vòng đời kết nối cơ bản.
- Định nghĩa ít nhất một Tool có schema đầu vào/đầu ra rõ; validate input và trả lỗi có ý nghĩa.
- Tích hợp với một MCP client thật và chứng minh client discover/call tool được.
- Giải thích ranh giới quyền: tool nào được phép làm gì; không cấp quyền rộng hơn nhu cầu.

Tầng 1B — Lỗi tự chọn theo mục tiêu ứng dụng

Không có số lượng cứng. Nhóm chọn các mục thực sự cần cho ứng dụng; đã chọn thì phải hiểu cơ chế, cấu hình và kết quả.

- Resources khi cần expose dữ liệu đọc; Prompts khi cần cung cấp prompt template.
- Transport phù hợp môi trường (stdio/HTTP-based nếu SDK hỗ trợ) và giải thích lựa chọn.
- Nhiều tools với domain khác nhau; grouping/metadata rõ.
- Lifespan/resource cleanup.
- Error mapping, timeout và retry ở client/server khi phù hợp.
- Auth/permission hoặc allowlist khi server truy cập dịch vụ nhạy cảm.
- Logging/tracing lời gọi tool.

Minh chứng tối thiểu cần có

- Ảnh/log client nhìn thấy tool/resource và gọi thành công ít nhất hai case (thành công + lỗi).
- Mã schema và handler của tool.
- Mô tả quyền/secret/data mà server có thể truy cập.

Chưa được xem là đạt nếu chỉ...

- Chỉ viết một REST API rồi gọi nó là MCP.
- Chỉ định nghĩa tool “echo/hello” không gắn với ứng dụng.
- Cho model quyền gọi shell/file/network quá rộng nhưng không có giới hạn.

Câu hỏi tự kiểm

- “Tool này khác một endpoint REST bình thường ở cách client/model discover và gọi thế nào?”
- “Nếu model truyền tham số sai schema thì server xử lý ra sao?”
- “Quyền tối thiểu mà tool này cần là gì?”

Ví dụ phạm vi có thể kiểm tra tại chỗ

Đây chỉ là ví dụ. Đề chính thức được ra từ chính đề án đã nộp, không hỏi kiến thức ngoài phạm vi nhóm đã làm.

- Thêm một tool mới có schema và error path.
- Thêm resource read-only hoặc giới hạn quyền của tool hiện có.
- Giải thích/debug một lời gọi tool thất bại từ client log.

faster-whisper — hồ sơ Speech-to-Text / Model Serving

Công nghệ chính: faster-whisper. Audio/video, phụ đề và UI chỉ là bối cảnh để chứng minh pipeline ASR và các đánh đổi tốc độ—chất lượng—tài nguyên.

Tầng 1A — Lỗi bắt buộc

- Quy trình đầu vào audio/video: trích xuất/chuẩn hóa audio phục vụ ASR, tránh tái mã hóa lossy không cần thiết.
- Tải và chạy model faster-whisper; hiểu vai trò của model size, device và compute_type.
- Sinh transcript theo segment với start/end/text và giải thích cấu trúc output.
- Cấu hình ngôn ngữ và tham số decoding cơ bản; hiểu ít nhất một ảnh hưởng tới tốc độ/chất lượng.
- Nhận biết lỗi ASR thường gặp: nhiễu, tên riêng, giọng nói, tốc độ nói, âm thanh kém.

Tầng 1B — Lỗi tự chọn theo mục tiêu ứng dụng

Không có số lượng cứng. Nhóm chọn các mục thực sự cần cho ứng dụng; đã chọn thì phải hiểu cơ chế, cấu hình và kết quả.

- Word-level timestamps nếu ứng dụng cần căn phụ đề chính xác.
- VAD nếu audio có nhiều khoảng im lặng.
- Chunking cho audio/video dài và ghép kết quả.
- Subtitle segmentation + xuất SRT/VTT/TXT nếu mục tiêu là phụ đề/biên bản.
- Language detection nếu nhiều ngôn ngữ.
- Batch processing nhiều file.
- Model serving/API nếu hệ thống cần tách model khỏi UI.

Minh chứng tối thiểu cần có

- Có ít nhất một file audio/video 10–60 phút chạy được từ đầu tới cuối.
- Có transcript chứa timestamp; nếu làm subtitle phải chứng minh SRT/VTT đúng thời gian cơ bản.
- Có ít nhất một phép đo: thời gian xử lý, real-time factor, WER/CER trên một mẫu có ground truth hoặc so sánh hai cấu hình.

Chưa được xem là đạt nếu chỉ...

- Chỉ gọi `transcribe()` và in text ra màn hình.
- Không giải thích model/device/compute_type hoặc timestamp.
- Chỉ làm UI upload/download nhưng không có đo lường hay phân tích ASR.

Câu hỏi tự kiểm

- “Đổi model size/compute_type thì đổi điều gì?”
- “Segment timestamp khác word timestamp ở đâu và ứng dụng của em cần loại nào?”
- “Khoảng lặng dài ảnh hưởng pipeline thế nào? VAD giúp gì?”

Ví dụ phạm vi có thể kiểm tra tại chỗ

Đây chỉ là ví dụ. Đề chính thức được ra từ chính đề án đã nộp, không hỏi kiến thức ngoài phạm vi nhóm đã làm.

- Thêm word-level timestamp và xuất JSON/SRT tương ứng.
- Bật/tinh chỉnh VAD cho một file có nhiều khoảng lặng và so sánh.
- Đổi hai cấu hình model/compute_type, đo lại thời gian và giải thích đánh đổi.

LlamaIndex — hồ sơ RAG

Công nghệ chính: LlamaIndex. RAG là kiến trúc ứng dụng; bộ tài liệu chỉ là corpus minh họa. Đề tài nên là hỏi đáp/tra cứu tài liệu, không biến thành một hệ chuyên gia nghiệp vụ khó như tư vấn pháp lý hoặc phân tích rủi ro pháp lý.

Tầng 1A — Lỗi bắt buộc

- Nạp dữ liệu vào LlamaIndex: Document/metadata và pipeline ingestion cơ bản.
- Chia tài liệu thành Node/chunk; giải thích chunk size/overlap ở mức ứng dụng.
- Embedding + lập chỉ mục; hiểu vector index lưu/tra cứu biểu diễn gì.
- Retriever: query → truy hồi các node liên quan; biết top-k và similarity ở mức sử dụng.
- Query/response synthesis: dùng context đã retrieve để tạo câu trả lời và giữ khả năng truy vết về nguồn.
- Có đánh giá thực nghiệm tối thiểu ở khâu retrieval hoặc answer grounding; không chỉ “thấy trả lời đúng”.

Tầng 1B — Lỗi tự chọn theo mục tiêu ứng dụng

Không có số lượng cứng. Nhóm chọn các mục thực sự cần cho ứng dụng; đã chọn thì phải hiểu cơ chế, cấu hình và kết quả.

- Metadata filtering theo loại tài liệu/chương/năm/đối tượng.
- Citation/source display và mở đoạn nguồn.
- Persistent index để không embedding lại toàn bộ mỗi lần chạy.
- Custom splitter/node parser phù hợp cấu trúc tài liệu.
- Nhiều nguồn dữ liệu hoặc nhiều collection.
- Conversation history/query transformation nếu cần hỏi tiếp.
- Reranking nếu retrieval ban đầu chưa đủ tốt.
- Incremental update/re-index khi thêm tài liệu.

Minh chứng tối thiểu cần có

- Vẽ pipeline thật của đồ án: ingestion → chunk/node → embedding/index → retrieval → context → generation → citation/evaluation.
- Cho thấy top-k retrieved nodes của một số câu hỏi, không chỉ hiện final answer.
- Có một bộ câu hỏi kiểm thử nhỏ và bảng kết quả retrieval/answer/source.

Chưa được xem là đạt nếu chỉ...

- Dán nguyên tài liệu dài vào prompt rồi hỏi model; đó không phải RAG.
- Gọi `VectorStoreIndex.from_documents()` + `query_engine.query()` nhưng không xem node/chunk/retrieval.
- Đánh giá bằng vài câu hỏi tự chọn ngay lúc demo rồi kết luận “chính xác”.
- Chọn miền nghiệp vụ quá khó (ví dụ kết luận rủi ro pháp lý) khiến nhóm phải học chuyên môn ngoài CNTT nhiều hơn học LlamaIndex.

Câu hỏi tự kiểm

- “Câu trả lời này được tạo từ những node nào?”
- “Nếu tăng chunk size/top-k thì retrieval thay đổi ra sao?”
- “Khi câu hỏi không có trong tài liệu, hệ thống phải làm gì?”

Ví dụ phạm vi có thể kiểm tra tại chỗ

Đây chỉ là ví dụ. Đề chính thức được ra từ chính đồ án đã nộp, không hỏi kiến thức ngoài phạm vi nhóm đã làm.

- Thay chunk size/top-k và so sánh retrieval cho cùng bộ câu hỏi.
- Thêm metadata filter/citation vào hệ thống hiện có.
- Tạo một câu hỏi ngoài corpus và chỉnh hệ thống để từ chối/giới hạn trả lời có căn cứ.

SDK mô hình — Gemini API / OpenAI / Claude

Đây là lớp gọi mô hình, thường là điểm khởi đầu chứ chưa phải một đề tài đủ sâu nếu đứng một mình.

Tầng 1A — Lỗi bắt buộc

- Gọi API đúng, quản lý API key an toàn và xử lý lỗi/timeout.
- Đặt tham số có chủ đích (model, temperature hoặc tham số tương đương, max output/token budget...) và giải thích tác động trong bài toán.
- Theo dõi token/chi phí hoặc ít nhất ước lượng được chi phí theo request.
- Thiết kế prompt có cấu trúc và kiểm tra output thay vì nối chuỗi tùy tiện.

Tầng 1B — Lỗi tự chọn theo mục tiêu ứng dụng

Không có số lượng cứng. Nhóm chọn các mục thực sự cần cho ứng dụng; đã chọn thì phải hiểu cơ chế, cấu hình và kết quả.

- Streaming response.
- Structured output/JSON schema.
- Function/tool calling.
- Multimodal input/output nếu ứng dụng cần.
- Cache theo request/query và rate limiting.
- Retry/backoff/fallback model.
- Prompt/version management và test regression.

Minh chứng tối thiểu cần có

- Có bảng cấu hình model/prompt thực tế và ít nhất một so sánh hai cấu hình hoặc hai model.
- Có log token/latency/cost cho một số request.
- Nếu dùng structured output/tool calling, có case sai schema/tool error.

Chưa được xem là đạt nếu chỉ...

- Chỉ gọi một API rồi in text ra giao diện.
- Không biết đang dùng model nào/param nào/cost ra sao.
- Để API key trong source code hoặc repo.

Câu hỏi tự kiểm

- “Tại sao chọn model này thay vì model nhỏ hơn/rẻ hơn?”
- “Temperature/structured output ảnh hưởng gì với tác vụ của em?”
- “Nếu API timeout/rate limit thì app xử lý thế nào?”

Ví dụ phạm vi có thể kiểm tra tại chỗ

Đây chỉ là ví dụ. Đề chính thức được ra từ chính đề án đã nộp, không hỏi kiến thức ngoài phạm vi nhóm đã làm.

- Thêm structured output cho một response đang là text tự do.
- Thêm retry/backoff hoặc fallback cho một lỗi API.
- Đo latency/cost hai model nếu nhóm đã làm tối ưu.

CẢNH BÁO — SDK MÔ HÌNH ĐÚNG MÌNH LÀ QUÁ NHẸ

SDK thường phải được ghép vào một hồ sơ rõ hơn như RAG, Agentic Workflow, Model Serving/Multimodal, hoặc phải có nghiên cứu so sánh/evaluation đủ sâu. “Gọi model + giao diện chat” không đủ để đại diện cho môn.

LangGraph / LangChain — hồ sơ điều phối workflow/agent

Nếu nhóm dùng LangGraph thì trọng tâm là stateful graph orchestration. Nếu dùng LangChain thuần thì phải chỉ rõ chain/runnable/tool/history nào là công nghệ chính; không được gọi chung “LangChain” nhưng đề án chỉ là một prompt nối model.

Tầng 1A — Lỗi bắt buộc

- Xác định rõ workflow/agent cần điều phối những bước nào và tại sao không thể chỉ gọi model một lần.
- Với LangGraph: State, Nodes, Edges và ít nhất một nhánh/điều kiện hoặc vòng lặp có ý nghĩa.
- Với LangChain thuần: chain/runnable có các bước rõ (prompt/model/parser/retriever/tool...) và data flow giữa các bước.
- Nếu có tool: schema, quyền và xử lý tool error phải rõ.
- Có cơ chế lưu trạng thái/history/persistence khi bài toán thật sự cần nhiều bước hoặc follow-up.

Tầng 1B — Lỗi tự chọn theo mục tiêu ứng dụng

Không có số lượng cứng. Nhóm chọn các mục thực sự cần cho ứng dụng; đã chọn thì phải hiểu cơ chế, cấu hình và kết quả.

- Conditional routing dựa trên output hoặc intent.
- Checkpoint/persistence và resume workflow.
- Human-in-the-loop tại bước nhạy cảm.
- Parallel branches hoặc subgraph/subchain.
- Multiple tools + tool selection.
- Timeout/max-step/loop guard để tránh agent chạy vô hạn.
- Tracing/observability workflow.

Minh chứng tối thiểu cần có

- Sơ đồ graph/chain đúng với code thật.
- Log ít nhất một run đi qua nhiều bước; nếu có nhánh phải có case đi qua hai nhánh khác nhau.
- Nếu dùng tool, có case tool thành công và tool thất bại.

Chưa được xem là đạt nếu chỉ...

- Một prompt → một model → một output nhưng gọi là “agent”.
- Dùng agent mặc định của thư viện mà không biết state/step/tool được điều phối ra sao.
- Cho agent quyền lặp/gọi tool không giới hạn.

Câu hỏi tự kiểm

- “State nào được giữ qua các node/step?”
- “Điều kiện nào quyết định đi nhánh A hay B?”
- “Nếu tool lỗi hoặc model cứ gọi tool lặp lại thì workflow dừng thế nào?”

Ví dụ phạm vi có thể kiểm tra tại chỗ

Đây chỉ là ví dụ. Đề chính thức được ra từ chính đề án đã nộp, không hỏi kiến thức ngoài phạm vi nhóm đã làm.

- Thêm một conditional branch vào graph/chain.
- Thêm checkpoint/history và resume một flow.
- Đặt max-step/timeout và xử lý tool error cho workflow hiện có.

Tầng 3 — Nâng cao chung cho Track Ứng dụng AI

Tầng 3 của AI nên ưu tiên “đo được” và “so sánh được”. Một đề án có 1 kỹ thuật nâng cao nhưng đánh giá sâu có thể tốt hơn đề án gán nhiều thư viện nhưng không có kiểm chứng.

- **Đánh giá có bộ test:** Tạo dataset câu hỏi/mẫu cố định; metric phù hợp từng hồ sơ (retrieval, WER/CER, mAP, latency, tool success, structured-output validity...).
- **So sánh cấu hình/model:** Thay model/param/chunking/rerank/quantization... và phân tích trade-off chất lượng–độ trễ–chi phí.
- **Retrieval nâng cao:** Hybrid search, rerank, query rewrite, router/multiple index; phải chứng minh cải thiện trên bộ câu hỏi.
- **Agent/tool nâng cao:** Giới hạn vòng lặp, quyền tool, human-in-the-loop, recovery khi tool/model lỗi.
- **Multimodal:** Kết hợp ảnh/audio/text khi thật sự phục vụ mục tiêu; có pipeline và đánh giá riêng.
- **Tối ưu inference:** Batching, quantization, ONNX/runtime, CPU/GPU comparison, caching; có benchmark.
- **Observability:** Trace một request qua các bước AI, lưu model/prompt/version/latency/error để tái hiện lỗi.
- **An toàn:** Prompt injection, data leakage, tool privilege, unsafe output; có ít nhất test case tấn công và biện pháp giảm thiểu.

4. Track Low-code / No-code

Định hướng track

Low-code/no-code không bị đánh giá thấp chỉ vì ít mã nguồn. Chiều sâu phải đến từ mô hình dữ liệu, biểu thức/rule, automation, security, offline/sync và khả năng giải thích luồng nền tảng. Không ép viết JavaScript/Python nếu nền tảng không cần.

Google AppSheet

Công nghệ chính là AppSheet. Một app vài bảng CRUD chưa đủ; nhóm phải khai thác cơ chế data model, expression, action/automation và security của nền tảng.

Tầng 1A — Lỗi bắt buộc

- Mô hình dữ liệu: Key, Ref, quan hệ giữa bảng và cách AppSheet dereference dữ liệu.
- Expression có ý nghĩa trong computed value/validation/show-if/filter...; không chỉ dùng expression mặc định.
- View và Action: tổ chức UI/interaction theo dữ liệu, biết action thay đổi row/navigation/process thế nào.
- Quyền truy cập dữ liệu cơ bản: biết người dùng nào nhìn/sửa được gì trong ứng dụng.

Tầng 1B — Lỗi tự chọn theo mục tiêu ứng dụng

Không có số lượng cứng. Nhóm chọn các mục thực sự cần cho ứng dụng; đã chọn thì phải hiểu cơ chế, cấu hình và kết quả.

- Slice để tạo tập dữ liệu/logic hiển thị có điều kiện.
- Bot/Automation với event, process, task và nhánh điều kiện.

- Security filter để giới hạn dữ liệu tải về theo user/role.
- Role-based view/action.
- Offline/sync và xử lý xung đột ở mức phù hợp.
- Form validation/required/valid-if/dependent dropdown.
- Báo cáo/dashboard/chart.
- Tích hợp Apps Script/API/webhook nếu ứng dụng thật sự cần mở rộng.

Minh chứng tối thiểu cần có

- Sơ đồ các bảng + Key/Ref và giải thích một quan hệ thật.
- Ít nhất vài expression do nhóm tự viết, kèm ảnh tình huống đúng/sai.
- Nếu dùng Bot/Security Filter/Offline, phải có case demo cụ thể chứ không chỉ bật tùy chọn.

Chưa được xem là đạt nếu chỉ...

- App hai bảng CRUD, dùng view/form mặc định và gần như không có expression.
- Dùng `Show\_If` để “ẩn nút” rồi xem như phân quyền; dữ liệu vẫn tải về đầy đủ.
- Tạo rất nhiều view nhưng logic dữ liệu/automation/security vẫn đơn giản.

Câu hỏi tự kiểm

- “Key khác Ref thế nào? Nếu đổi key có hậu quả gì?”
- “Security filter khác Slice ở điểm nào?”
- “Bot này được kích hoạt bởi event gì và nếu task lỗi thì em biết ở đâu?”

Ví dụ phạm vi có thể kiểm tra tại chỗ

Đây chỉ là ví dụ. Đề chính thức được ra từ chính đề án đã nộp, không hỏi kiến thức ngoài phạm vi nhóm đã làm.

- Thêm một Ref và dereference vào luồng đang có.
- Thêm security filter/role rule cho một bảng.
- Thêm một Bot có điều kiện và chứng minh hai nhánh khác nhau.

CẢNH BÁO — APPSHEET DỄ “QUÁ NHẸ”

Chiều sâu không đến từ số lượng bảng/màn hình mà từ data model, expression, validation, automation, security, offline/sync và integration. Nhóm không cần viết code nếu nền tảng giải quyết được bằng cơ chế nguyên bản.

5. Checklist nhóm phải hoàn thành trước khi nộp

1. Ghi đúng tên công nghệ chính trong tên đề tài và báo cáo; không chỉ ghi tên bài toán nghiệp vụ.
2. Đánh dấu các mục Tầng 1A đã hoàn thành và chỉ rõ file/module/route/screen nơi thể hiện.
3. Liệt kê các mục Tầng 1B nhóm chủ động chọn vì phù hợp ứng dụng; nêu lý do chọn.
4. Liệt kê các yêu cầu Tầng 2 đã thực hiện và minh chứng chạy/test/repo.
5. Liệt kê Tầng 3 nếu có; mỗi mục phải kèm số liệu, test case hoặc minh chứng kỹ thuật tương xứng.
6. Bảng phủ kỹ thuật trong báo cáo phải khớp code đã nộp; không khai kỹ thuật chỉ có trong slide hoặc demo rời.
7. Chuẩn bị để bất kỳ thành viên nào cũng có thể tìm đúng code, sửa một yêu cầu nhỏ và giải thích cơ chế mà không dùng AI.
8. Kiểm tra lại phạm vi nghiệp vụ: nếu nhóm vẫn dành phần lớn thời gian cho quy trình/dữ liệu nghiệp vụ thay vì công nghệ, hãy đơn giản hóa trước khi quá muộn.

NHỚ LẠI MỘT CÂU

Bài toán chỉ cần đủ đơn giản để công nghệ có chỗ thể hiện. Điểm cao đến từ việc hiểu đúng, dùng đúng và đào sâu công nghệ — không đến từ việc xây một “hệ thống nghiệp vụ thật” thật lớn.